

Feedforward Neural Nets

Overview

[date]

- identify and describe the different types of non-linearities in deep learning systems
- design and implement multi-layer perceptrons (MLPs) for complex problems

Async

[optional]

Feedforward Networks and the Need for Non-Linearities

Linear Models

- so far we have seen two linear models

- linear regression

$$y = w_T x$$

- logistic regression

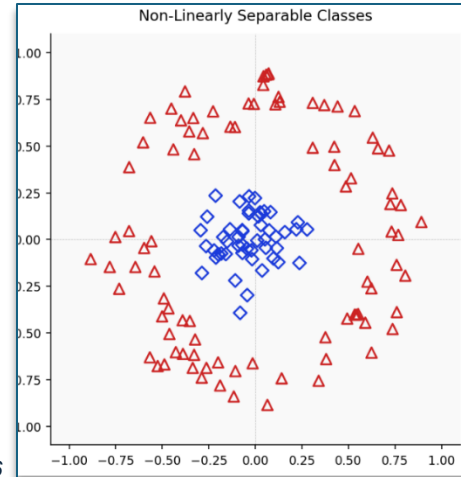
$$P(y = k) \propto \exp(w_T x)$$

What is Model Linearity?

- an increase in a feature value x_k either always increases or decreases the score
 - depending on the sign of w_k
- effect of change in feature value of x_k on score is same regardless of value of x_k
 - example:
 - the change in house price for $1br \rightarrow 2br$ is the same as from $9br \rightarrow 10br$

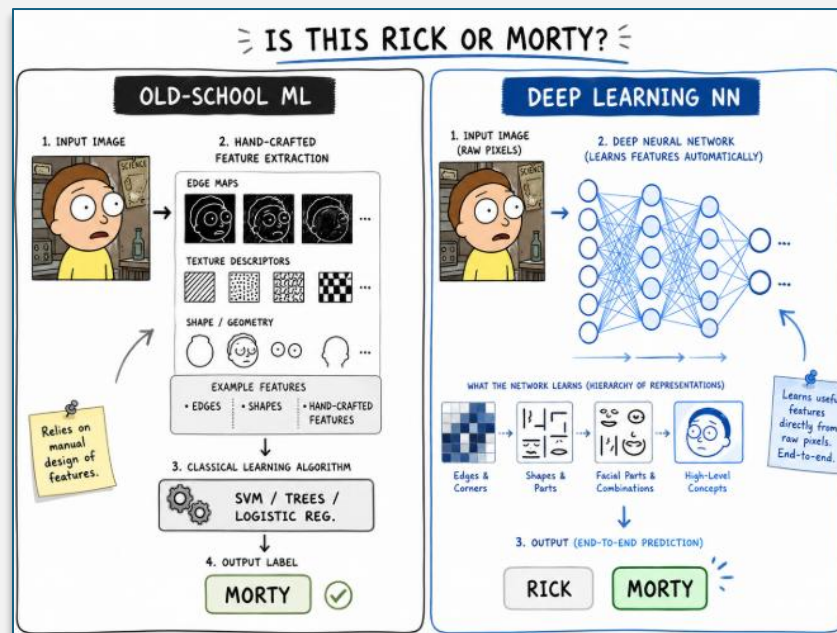
Why We Want Nonlinearities

- traditional ML approaches deal with non-linear scenarios using complex methods (ex: kernels) or heavy feature engineering
- however, neural networks give us a different approach using nonlinear functions



Sonnet 4.6

“Old School” ML vs Deep Learning NN

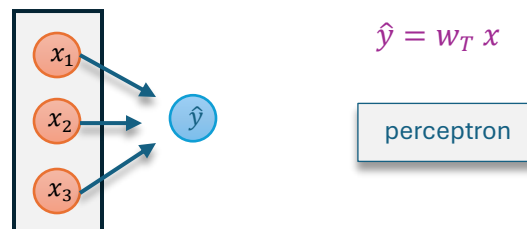


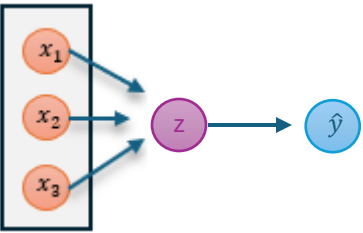
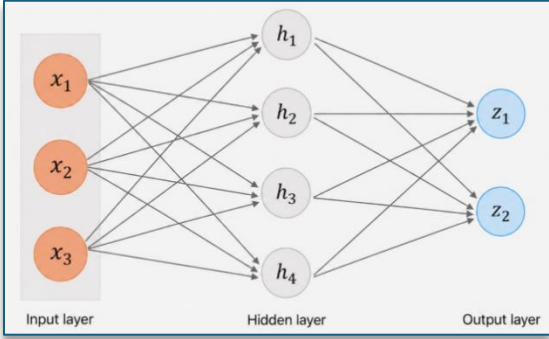
ChatGPT 5.4

Architecture of FFNs

Linear Regression as a NN

- you can think of linear regression as a one-layer NN



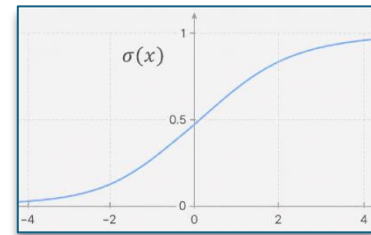
Binary Logistic Regression	$z = w_T x \quad \hat{y} = \text{sigmoid}(z)$  perceptron with a sigmoid / softmax
Multilayer Perceptron (MLP)	$H = X W_1$ - a composition of linear functions is still a linear function - no need to apply nonlinearities! 
Why We Want Nonlinearities	- one "layer" in a NN is a linear transformation followed by a non-linearity - nonlinearities are usually element-wise operations - that is, $h = \sigma(x)$ - so, $h_i = \sigma(x_i)$

Activation Functions

Sigmoid (x)

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

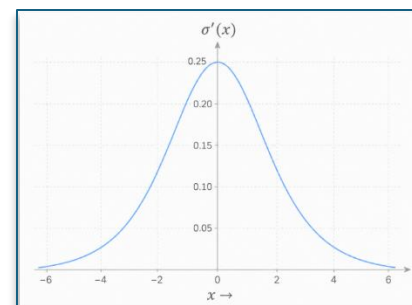
- used to squash real value to probability



$$\frac{d}{dx} \sigma(x) = \frac{-1}{(1 + \exp(-x))^2} \frac{d}{dx} (1 + \exp(-x))$$

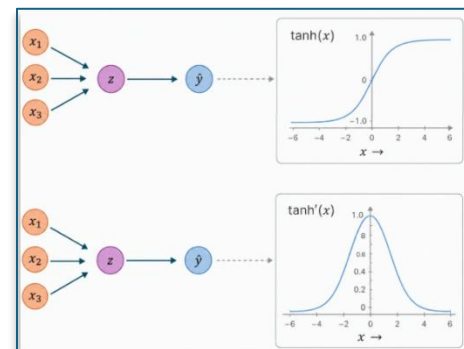
$$= \frac{\exp(-x)}{(1 + \exp(-x))^2}$$

$$= \sigma(x)(1 - \sigma(x))$$

**Tanh (x)**

$$\tanh(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

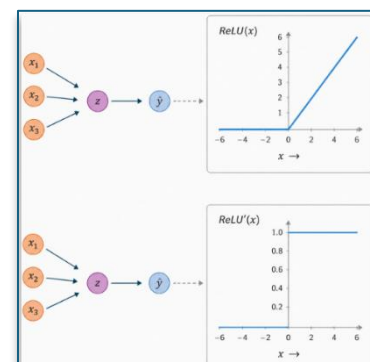
**ReLU (x)**

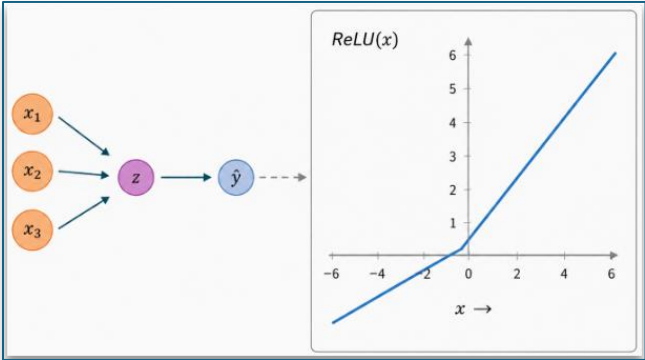
- “rectified linear unit”

$$\text{ReLU}(x) = \max(0, x)$$

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$$

- much faster than sigmoid / tanh



Leaky ReLU (x)	$\text{Leaky ReLU}(x) = \max(0.01x, x)$ $\frac{d}{dx} \text{Leaky ReLU}(x) = \begin{cases} 0.01, & x < 0 \\ 1, & x \geq 0 \end{cases}$  The diagram shows three input nodes x_1, x_2, x_3 (orange circles) feeding into a hidden node z (purple circle). An arrow from z points to an output node $\hat{y}$ (blue circle). To the right, a graph of the ReLU function $\text{ReLU}(x)$ is shown, with the x-axis ranging from -6 to 6 and the y-axis from 0 to 6. The function is zero for $x \leq 0$ and increases linearly for $x > 0$.
Advice in Practice	• try ReLU out first • usually better and faster to train than sigmoid or tanh activations • might need to be careful with learning rates and instantiation • Leaky ReLU and other modifications can lead to some further gains
Universal Approximation Theorem	
UAT Definition	• a neural network can approximate any function arbitrarily well, if it contains: ○ at least one hidden layer ○ sufficient width ○ a nonlinear activation function
Implications	• NNs can overfit any complex dataset ○ needs regularization! • complex functions may require large NNs ○ explore different architectures

- how do we find a network that approximates a desired function?
 - backpropagation

Implementing FFNs

Python

